POLITECNICO
MILANO 1863

Ingegneria del software

Design pattern strutturali

Decorator

Motivazioni

- Consideriamo un sistema per creare interfacce grafiche
- Consideriamo una classe *TextView* rappresentante una finestra contenente del testo
- TextView può essere arricchita con delle barre di scorrimento se il testo contenuto supera una certa lunghezza
- TextView può essere arricchita con dei bordi per migliorare la leggibilità del testo

Motivazioni

- **Problema:** arricchire entità con «decorazioni» diverse
- *Non tutte le TextView* necessitano di barre di scorrimento e bordi: alcune possono necessitare solo di bordi e altre di nulla.
- Le decorazioni possono essere molteplici e potenzialmente estendibili

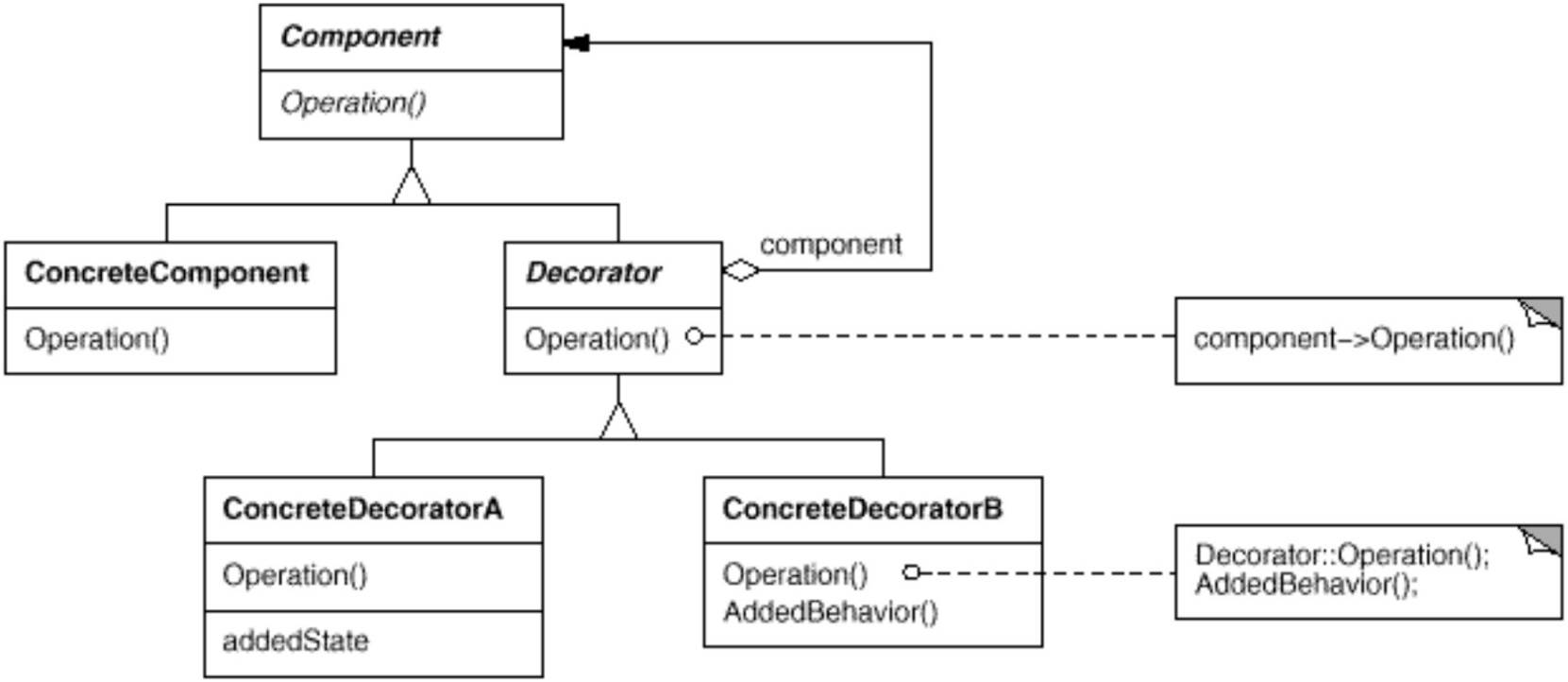
Una soluzione non ideale

- Generare una sottoclasse per ogni decorazione da aggiungere
- Ma anche per ciascuna combinazione possibile di decorazioni
- TextView, TexViewWithBorders, TextViewWithScrollbars, TextViewWithBordersAndScrollbars, …

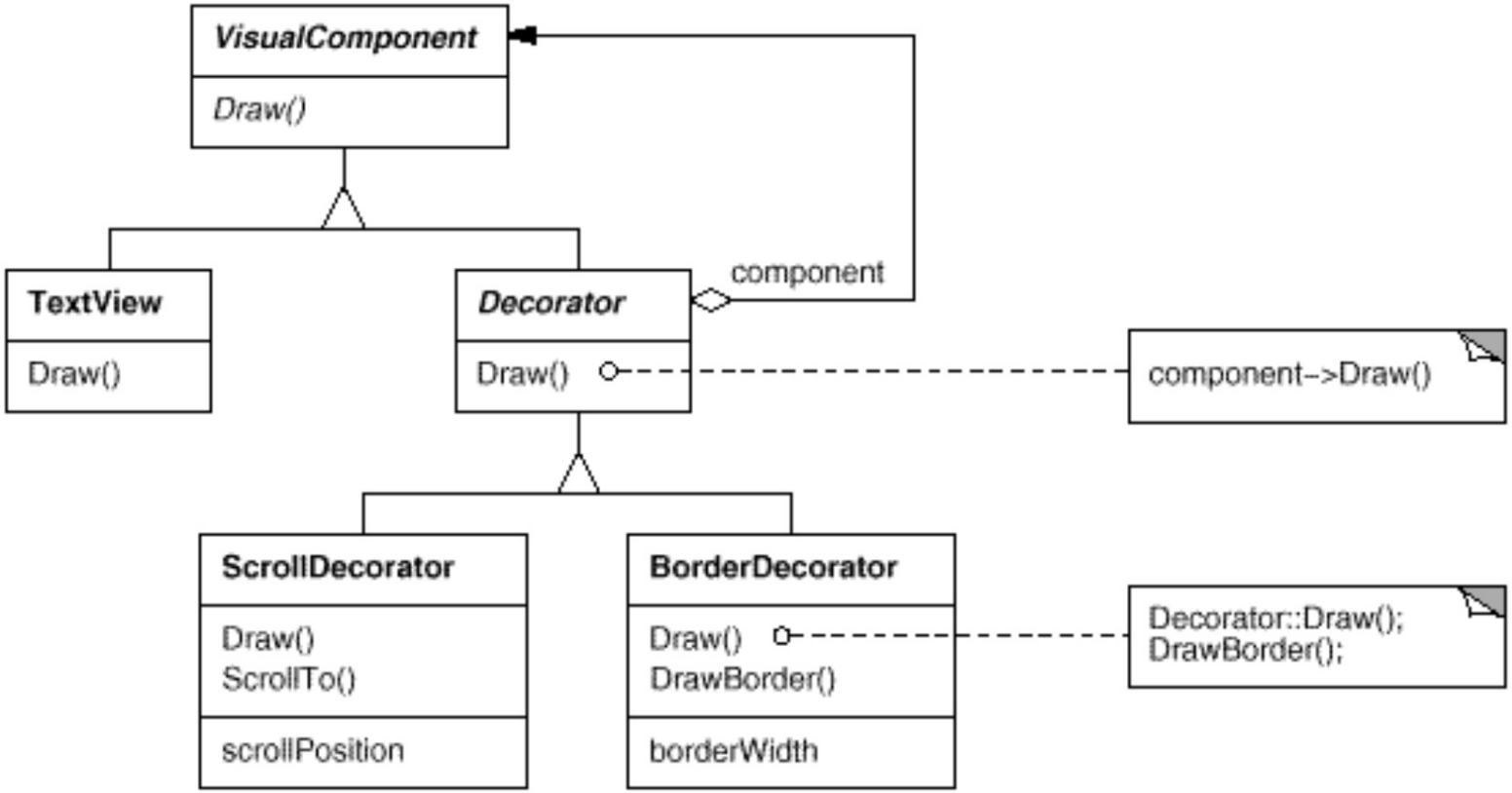
Obbiettivi

- Definire una classe astratta «decorator» che implementa la stessa astrazione della classe
- La classe decoratrice mantiene un riferimento all'oggetto da decorare
- Implementazioni della classe astratta implementano decorazioni diverse
- È possibile decorare un oggetto decorato

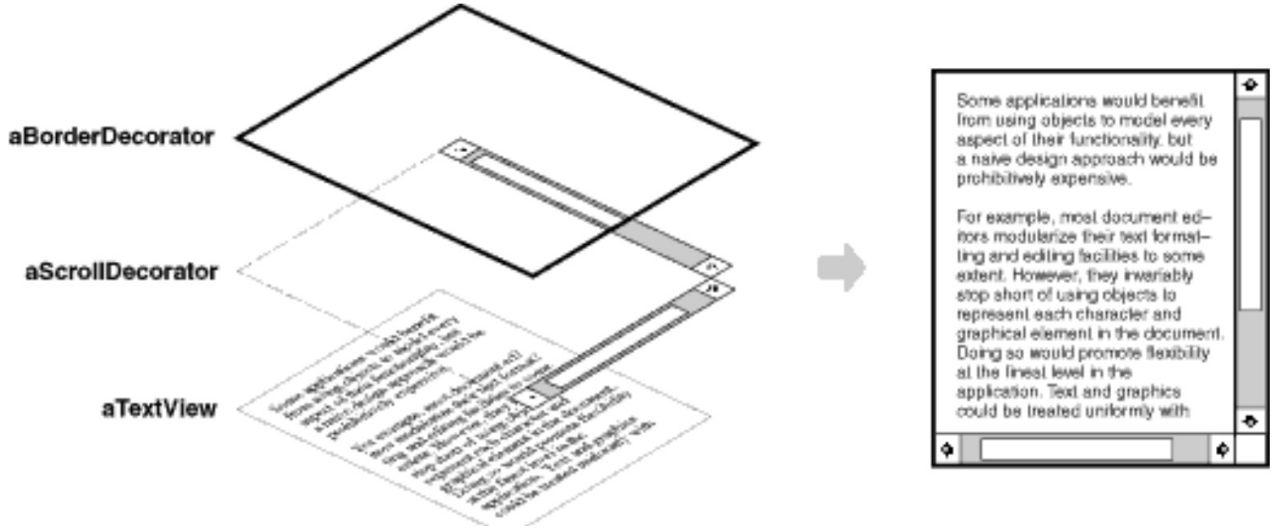
Struttura

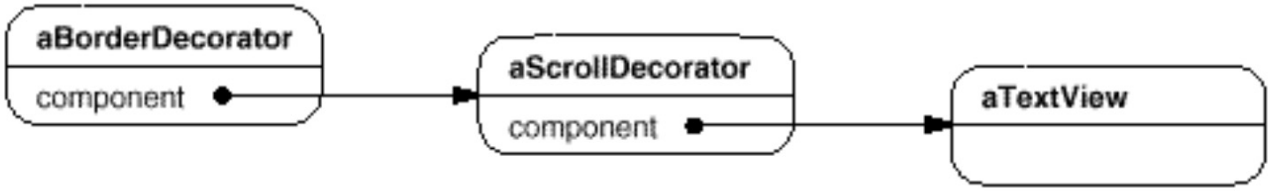

Component
Operation()
ConcreteComponent
Operation()
Decorator
Operation()
component
component->Operation()
ConcreteDecoratorA
Operation()
addedState
ConcreteDecoratorB
Operation()
AddedBehavior()
Decorator::Operation();
AddedBehavior();

Esempio


VisualComponent
Draw()
TextView
Draw()
Decorator
Draw()
component
component->Draw()
ScrollDecorator
Draw()
ScrollTo()
scrollPosition
BorderDecorator
Draw()
DrawBorder()
borderWidth
Decorator::Draw();
DrawBorder();

Esempio


aBorderDecorator
aScrollDecorator
aTextView
Some applications would benefit from using objects to model every aspect of their functionality, but a naive design approach would be prohibitively expensive.
For example, most document editors modularize their text formatting and editing facilities to some extent. However, they invariably stop short of using objects to represent each character and graphical element in the document. Doing so would promote flexibility at the finest level in the application. Text and graphics could be treated uniformly with


aBorderDecorator
component
aScrollDecorator
component
aTextView

Esempio

```
interface VisualComponent {
      void draw();
      …
}


class TextView implements VisualComponent {
      public void draw(){
          …
      }
}
```

Esempio

```
abstract class Decorator implements VisualComponent {
        private VisualComponent component;

        public Decorator(VisualComponent c){
                component = c;
        }

        public void draw(){
            component.draw()
        }
        …
}
```

Esempio

```
class BorderDecorator extends Decorator {

        public BorderDecorator(VisualComponent c){
                super(c);
        }

        public void draw(){
            super.draw();
            drawBorder();
        }

        private void drawBorder(){
                …
        }

}
```

Esempio

```
class Client {

        public void setup(){
                TextView t = new TextView();
                VisualComponent v = new BorderDecorator(t);
                VisualComponent s = new ScrollDecorator(v);
                s.draw();
        }
}
```